Week 7.2: Set

Introduction to Algorithms and Data Structures

SAXION UNIVERSITY OF APPLIED SCIENCES

We would like some (anonymous) feedback on this module!

edu.nl/acpfv

Set

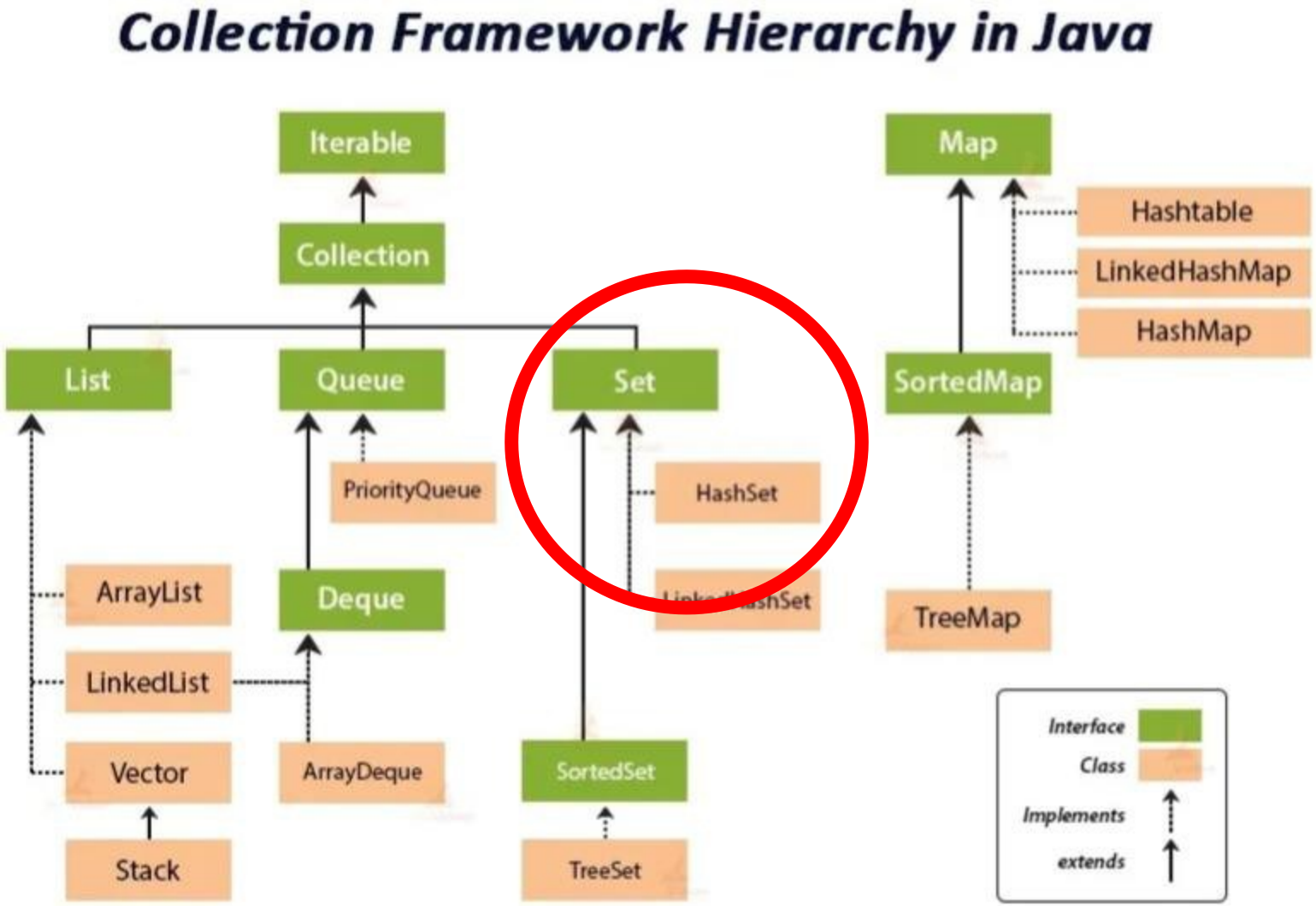
- **Discuss work last lesson**
- Set and HashSet
- Using HashSet
- Summary
- Homework

- Where you able to do the exercises?
  1. Yes! All exercises are done!
  2. No.., Because I have these questions
  3. I haven't prioritized this subject enough at the moment to do the assignments

Set

- **Discuss work last lesson**
- **Set and HashSet**
- Using HashSet
- Summary
- Homework

Set interface



Most important concepts about a Set

- **Set** (**interface**) is a specialized collection, a bit like List:
  - it is comparable to a set in Mathematics;
    - **No duplicate elements allowed** (adding duplicates is ignored)
    - **No guaranteed order**
  - it has **no direct retrieval** (no `get()`), but a check if it is there (**`contains()`**),
    - but it is iterable, so you can loop over it with an enhanced for-loop

Set interface

<<Java Interface>>

Set<E>

java.util

- size():int
- isEmpty():boolean
- contains(Object):boolean
- iterator():Iterator<E>
- toArray():Object[]
- toArray(T[]):T[]
- add(E):boolean
- remove(Object):boolean
- containsAll(Collection<?>):boolean
- addAll(Collection<? extends E>):boolean
- retainAll(Collection<?>):boolean
- removeAll(Collection<?>):boolean
- clear():void
- equals(Object):boolean
- hashCode():int
- spliterator():Spliterator<E>

HashSet, a Set implementation

- Utilizes ‘hashing’ to (swiftly) generate a unique integer value for any element added, serving as an index, just as with HashMap keys;
  - uses **`Object.hashCode()`** method to generate a has (unique integer) and
  - so fast check if an element is part of the set.

Set

- **Discuss work last lesson**
- **Set and HashSet**
- **Using HashSet**
- Summary
- Homework

Adding elements

Set the type of objects stored

```
HashSet<String> mySet = new HashSet<String>();
mySet.add("Netherlands");
mySet.add("Germany");
mySet.add("China");

System.out.print(mySet);
```

Add elements

[Netherlands, China, Germany]

Adding elements

```
HashSet<String> mySet = new HashSet<String>();
mySet.add("Netherlands");
mySet.add("Germany");
mySet.add("China");
mySet.add("China");

System.out.print(mySet);
```

[Netherlands, China, Germany]

NOTE: China is added twice but only once in the set!

NOTE: Adding it multiple times will NOT crash your program!

Removing elements

```
HashSet<String> mySet = new HashSet<String>();
mySet.add("Netherlands");
mySet.add("Germany");
mySet.add("China");
mySet.remove("China");

System.out.print(mySet);
```

Remove an element

[Netherlands, Germany]

Removing elements – what happens now?

```
HashSet<String> mySet = new HashSet<String>();
mySet.add("Netherlands");
mySet.add("Germany");
mySet.add("China");
mySet.add("China");
mySet.remove("China");

System.out.print(mySet);
```

Remove an element

[Netherlands, Germany]

Looping over elements

NOTE: Remember there are no guarantees in which order the elements are stored/printed!

```
HashSet<String> mySet = new HashSet<String>();
mySet.add("Netherlands");
mySet.add("Germany");
mySet.add("China");

for (String country : mySet) {
    System.out.println(country);
}
```

Loop over elements

Output:

Netherlands
China
Germany

Some handy functions

```
HashSet<String> mySet = new HashSet<String>();
mySet.add("Netherlands");
mySet.add("Germany");
mySet.add("China");
mySet.add("China");

mySet.contains("Germany");        //result: true
mySet.contains("Spain");          //result: false
mySet.size();                     //result: 3
```

How to work with objects (reference types)

```
class MyClass{
    int nr;
    public MyClass(int nr) {
        this.nr = nr;
    }
}
```

```
HashSet<MyClass> mySet = new HashSet<MyClass>();

MyClass myObject = new MyClass(1);
mySet.add(myObject);

mySet.add(new MyClass(2));                          //can add directly

mySet.contains(myObject); //true or false?          //true
mySet.contains(new MyClass(2)); //true or false?    //false!
```

The `Collections.contains()` checks for an element by using the `Object.equals()` method, which by default tests for **reference equality using `==`**, although String overrides this by checking on content (characters).
To change how elements are compared (for example, based on their content), overriding the '**equals**` and '**hashCode**' method in MyClass.

Set

- **Discuss work last lesson**
- **Set and HashSet**
- **Using HashSet**
- **Summary**
- Homework

Summary (Hash)Set

- **Set** (**interface**) is a specialized collection, comparable to a set in Mathematics;
  - **No duplicate elements allowed** (adding duplicates is ignored).
  - **No guaranteed order.**
  - **No direct retrieval but check on containment** and iteration over elements.
- **HashSet** (**implementation**) is a Set implementation using hashing;
  - Override **`equals()`** and **`hashCode()`** on the elements class.

Set

- Discuss work last lesson
- Set and HashSet
- Using HashSet
- Summary
- Homework

Now it’s your turn!